

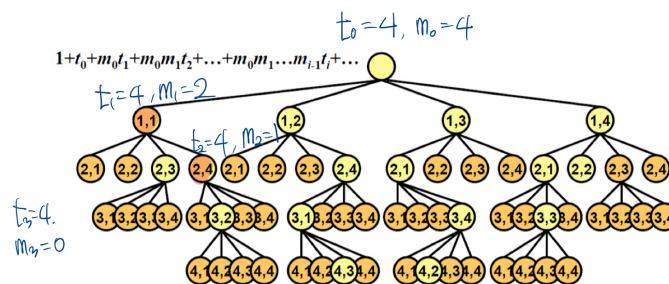
Hands-on assignment 3

2466025 박혜린

3-1: Monte Carlo Simulation for n-queens problem

Compute the number of nodes expanded when selecting (1,1) and (2,4)?

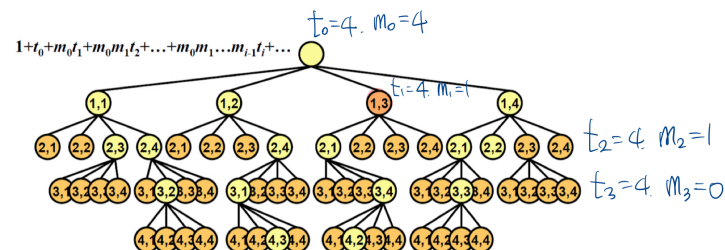
Q1.



$$\begin{aligned}
 & 1 + t_0 + m_0 t_0 + m_0 m_1 t_2 + m_0 m_1 m_2 t_3 \\
 &= 1 + 4 + 4 \times 4 + 4 \times 2 \times 4 + 4 \times 2 \times 1 \times 4 \\
 &= 1 + 4 + 16 + 32 + 32 = 85
 \end{aligned}$$

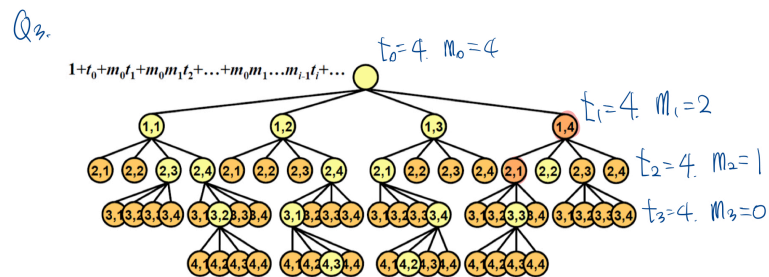
Compute the number of nodes expanded when selecting (1,3)?

Q2.



$$\begin{aligned}
 & 1 + t_0 + m_0 t_0 + m_0 m_1 t_2 + m_0 m_1 m_2 t_3 \\
 &= 1 + 4 + 4 \times 4 + 4 \times 1 \times 4 + 4 \times 1 \times 1 \times 4 \\
 &= 1 + 4 + 16 + 16 + 16 = 53
 \end{aligned}$$

Compute the number of nodes expanded when selecting (1,4) and (2,1)?



$$\begin{aligned}
 & 1 + t_0 + m_0 t_0 + m_0 m_1 t_1 + m_0 m_1 m_2 t_2 \\
 &= 1 + 4 + 4 \times 4 + 4 \times 2 \times 4 + 4 \times 2 \times 1 \times 4 \\
 &= 1 + 4 + 16 + 32 + 32 \\
 &= 85
 \end{aligned}$$

Subset Sum

Step 1. Prompt an AI to solve this exact problem using backtracking.

Prompt

Use the backtracking algorithm to solve the subset sum problem, $n = 6, w_1 = 3, w_2 = 34, w_3 = 4, w_4 = 12, w_5 = 5, w_6 = 2, M = 9$
generate code in python



Use the backtracking algorithm to solve the subset sum problem, $n = 6, w_1 = 3, w_2 = 34, w_3 = 4, w_4 = 12, w_5 = 5, w_6 = 2, M = 9$
generate code in python

<> Python



▶ 실행됨

```
# Backtracking solution for the Subset Sum Problem
# n = 6
# weights = [3, 34, 4, 12, 5, 2]
# target M = 9

weights = [3, 34, 4, 12, 5, 2]
target = 9

solutions = []

def subset_sum(index, current_sum, subset):
    # Case 1: target found
    if current_sum == target:
        solutions.append(subset[:])
        return

    # Case 2: out of bounds or exceeded target
    if index >= len(weights) or current_sum > target:
```



+ 무엇이든 물어보세요



ChatGPT는 실수를 할 수 있습니다. 중요한 정보는 재차 확인하세요.

```
</> Python 실행됨

# Case 2: out of bounds or exceeded target
if index >= len(weights) or current_sum > target:
    return

# Include current element
subset.append(weights[index])
subset_sum(index + 1,
           current_sum + weights[index],
           subset)

# Backtrack (exclude current element)
subset.pop()
subset_sum(index + 1,
           current_sum,
           subset)

# Run backtracking
subset_sum(0, 0, [])

# Print results
print("Subsets with sum", target, ":")

for s in solutions:
    print(s)
```

Output

```
</> Python 실행됨

Subsets with sum 9 :
[3, 4, 2]
[4, 5]
```

+ 무엇이든 물어보세요



ChatGPT는 실수를 할 수 있습니다. 중요한 정보는 재차 확인하세요.

AI's generated code

```
# Backtracking solution for the Subset Sum Problem
# n = 6
# weights = [3, 34, 4, 12, 5, 2]
# target M = 9
```

```

weights = [3, 34, 4, 12, 5, 2]
target = 9

solutions = []

def subset_sum(index, current_sum, subset):
    # Case 1: target found
    if current_sum == target:
        solutions.append(subset[:])
        return

    # Case 2: out of bounds or exceeded target
    if index >= len(weights) or current_sum > target:
        return

    # Include current element
    subset.append(weights[index])
    subset_sum(index + 1,
               current_sum + weights[index],
               subset)

    # Backtrack (exclude current element)
    subset.pop()
    subset_sum(index + 1,
               current_sum,
               subset)

# Run backtracking
subset_sum(0, 0, [])

# Print results
print("Subsets with sum", target, ":")

for s in solutions:
    print(s)

```

```
[Running] python -u "C:\Users\hyeri\AppData\Local\Temp\tempCodeRunnerFile.python"
Subsets with sum 9 :
[3, 4, 2]
[4, 5]

[Done] exited with code=0 in 0.596 seconds
```

Manually modified code

```
# Backtracking solution for the Subset Sum Problem
# n = 6
# weights = [3, 34, 4, 12, 5, 2]
# target M = 9

weights = [3, 34, 4, 12, 5, 2]
target = 9

solutions = []

node_count = 0 ### generated manually

def subset_sum(index, current_sum, subset):
    ### generated manually
    global node_count
    node_count += 1
    ###

    # Case 1: target found
    if current_sum == target:
        solutions.append(subset[:])
        return

    # Case 2: out of bounds or exceeded target
    if index >= len(weights) or current_sum > target:
        return

    # Include current element
    subset.append(weights[index])
```

```

subset_sum(index + 1,
            current_sum + weights[index],
            subset)

# Backtrack (exclude current element)
subset.pop()
subset_sum(index + 1, current_sum, subset)

# Run backtracking
subset_sum(0, 0, [])

# Print results
print("Subsets with sum", target)

print(f"Nodes visited : {node_count}") ### generated manually

for s in solutions:
    print(s)

```

```

[Running] python -u
"C:\Users\hyeri\AppData\Local\Temp\tempCodeRunnerFile.python"
Subsets with sum 9
Nodes visited : 39
[3, 4, 2]
[4, 5]

```

Step 2. AI models often write naive recursive functions that mimic exhaustive search $O(2^n)$ because they miss strict bounding conditions and fail to pre-sort the inputs. Analyze the AI's code and point out what specific pruning logic it missed.

1. Weak Existing Conditions

```

if index >= len(weights) or current_sum > target:

```

```
return
```

`index >= len(weights)` : stopping at leaf nodes is just base case recursion, not pruning

`current_sum > target` : Pruning only when the sum is already exceeds. Therefore, this is not a pre-check but a post-inclusion check.

2. Missing pre-check

Also, there is no remaining-sum feasibility check before recursing. Without this, the algorithm always explores exclude branch even when the remaining elements make it mathematically impossible to reach the target.

3. Missing duplicate prevention

There is no logic to prevent duplicate searches when multiple elements have the same value. For example, if `weights = [2, 2, 3, 4, 5]` and `target = 9`, selecting first `2` and selecting second `2` produces the exact same subset, yet this code explores both paths as if they were distinct.

Step 3. Fix the AI's code based on the pseudo code in Slide 20.

```
node_count = 0
prune_count = 0

def subset_sum(weights, target):
    global node_count, prune_count
    node_count = 0
    prune_count = 0

    ### generated manually
    weights.sort() # sort the weights in ascending order
    # Crucial for the bounding condition (wsum + weights[index] > target) to prune efficiently.
    solutions = []

    # wsum: sum of the elements selected so far
```

```

    # rsum: sum of the remaining elements after the current
index
    total = sum(weights) # total: total sum of all elements
(used as the initial 'rsum')

    # divide subset_sum into is_expand and backtrack
    # Bounding function
    def is_expand(index, wsum, rsum):
        # Condition 1: wsum + rsum >= M
        # Even if all remaining values are added, the sum i
s still less than the target -> stop searching
        condition1 = (wsum + rsum >= target)

        # Condition 2: wsum + w[i+1] <= M
        # Adding the next element exceeds the target -> sto
p searching
        condition2 = (index < len(weights) and wsum + weigh
ts[index] <= target)
        return condition1 and condition2
    ###

    ### added rsum manually based on pseudo code on page 20
    def backtrack(index, wsum, rsum, subset):
        global node_count, prune_count
        # count every recursive call as a visited node in t
he State Space Tree
        node_count += 1

        # base case: reached target sum accurately
        if wsum == target: ### changed name current_sum ->
wsum
            solutions.append(list(subset))
            return

        # Reached the end of the weights => No more element
s to explore
        if index >= len(weights):
            return

```

```

    ### generated manually
    # Check is_expand: determine whether further exploration is worthwhile or should be pruned based on bounding conditions
    if not is_expand(index, wsum, rsum):
        prune_count += 1 # Count the branch as pruned
        return
    ###

    # Left Child Node: Include the current element (weights[index])
    subset.append(weights[index])
    backtrack(index + 1, wsum + weights[index], rsum - weights[index], subset) # Update wsum (increases) and rsum (decreases) by selected weights

    # right node: Exclude the current element (weights[index])
    subset.pop()
    backtrack(index + 1, wsum, rsum - weights[index], subset) # wsum stays the same, rsum decreases (by passed weights)

    # Start backtracking from index 0, current sum 0, remaining sum 'total', and an empty subset
    backtrack(0, 0, total, [])
    return solutions

# Input Data
weights = [3, 34, 4, 12, 5, 2]
target = 9

solutions = subset_sum(weights, target)

print(f"Given weights: {weights}")
print(f"Target sum (M): {target}\n")

```

```

print(f"Found {len(solutions)} solution(s):")
for s in solutions:
    print(s)

print(f"Nodes visited : {node_count}")
print(f"Branches pruned: {prune_count}")

```

```

[Running] python -u "c:\Users\hyeri\OneDrive\Desktop\ha3_2.py"
Given weights: [2, 3, 4, 5, 12, 34]
Target sum (M): 9

Found 2 solution(s):
[2, 3, 4]
[4, 5]
Nodes visited : 23
Branches pruned: 10

[Done] exited with code=0 in 0.783 seconds

```

1. Redesign function structure

- Restructured from a single global recursive function `subset_sum` to a wrapper function containing two nested functions: `is_expand()`, `backtrack()`
- Moved `weights`, `target`, `solutions` from global to local(function) parameters

2. Adding parameter `rsum`

Added parameter which tracks the sum of remaining elements for bounding condition 1

3. Adding Bounding function `is_expand`

- **Condition 1:** `wsum + rsum >= target` (lower bound)

`wsum` : sum of elements selected so far

`rsum` : sum of all remaining elements from the current index to the end, passed as a parameter and updated at each recursive call by subtracting the current element. (retrieved in $O(1)$)

If selecting all remaining elements are impossible to reach the target value M , then no valid solution can exist along this branch. Therefore, the search can be safely terminated.

- **Condition 2:** $wsum + weights[index] \leq target$ (upper bound)

$weights[index]$: the smallest remaining element (since the weight is sorted)

Even if adding even the smallest remaining element to $wsum$ already exceeds M , then every subsequent element will also exceed M since they are larger. Therefore, further exploration of this branch is not needed anymore.

4. Adding `weights.sort()`

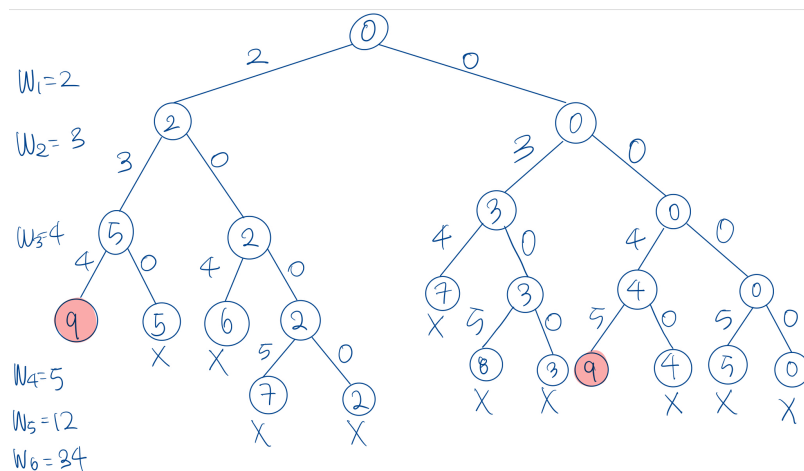
Sorting weights in ascending order to improve the pruning efficiency of condition 2. Sorting makes upper bound check to trigger as early as possible

Weights are sorted in ascending order to improve the pruning efficiency of Condition 2. Sorting ensures that $weights[index]$ is always the smallest remaining element, allowing the upper bound check to trigger as early as possible.

5. Adding `prune_count`

Adding a counter to track the number of branches pruned during the search.

Step 4. Manually draw the highly pruned state space tree based on your optimized code from Step 3. Briefly discuss how pre-sorting the weights mathematically guaranteed that you could cut off certain branches earlier than the AI's initial approach.



After sorting, $w[0] \leq w[1] \leq \dots \leq w[n-1]$ holds. Therefore, if $wsum + w[i] > M$, then every subsequent element, which is greater than or equal to $w[i]$ must exceed M . Due to this monotonicity property, a single `break` can prune the entire remaining branch. Without pre-sorting, it is impossible to guarantee $w[i]$ is the smallest remaining element. Therefore, this logic no longer holds and it requires continued exploration because a smaller element could appear later.